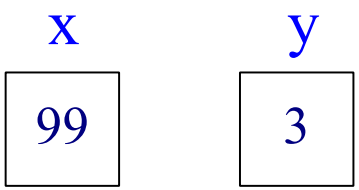
Namen und Datentypen

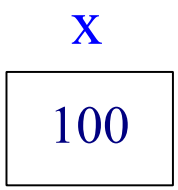
Prozedurale Programmierung: Begriff der Variablen

- Variablen sind eine Art benannte Behälter für Werte

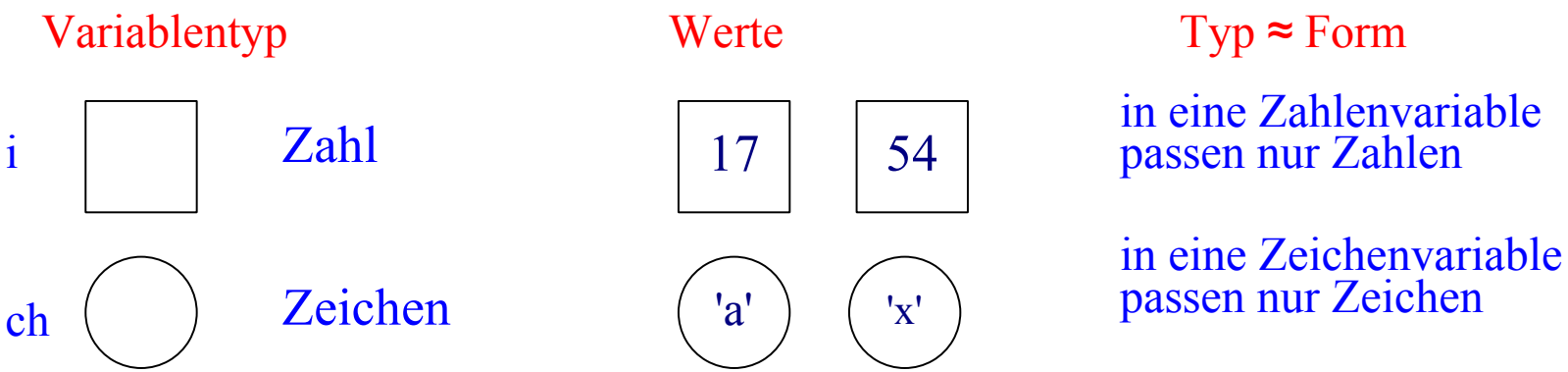


- können ihren Wert ändern

x := x +1



- haben einen Datentyp
  - beschreibt Menge erlaubter Werte + Operatoren



Begriff der Variablen: Programmiertechnisch

- Variablen werden vom Compiler (Übersetzer) auf Speicherzellen abgebildet
- Eine ***Variable*** hat
  - einen ***Namen***
    - über den die Variable im Programm angesprochen wird
  - eine ***Adresse***
    - zugewiesener Speicherbereich
      - erfolgt in imperativen Sprachen zur Übersetzungszeit
      - Größe des notwendigen Speicherbereichs ist durch Datentyp festgelegt
    - symbolisiert durch den Namen der Variable
  - einen ***Wert***
    - ist durch Auswertung eines Ausdrucks gegeben
    - codiert als Binärsequenz
  - einen eindeutigen ***Datentyp***

Namen

Namen in Programmiersprachen

- ***Namen*** (oder Bezeichner) werden benutzt
  - zur Benennung von Variablen, Typen, Funktionen, etc.
- ***lexikalische Regeln*** definieren Namensverwendung
  - stehen (oftmals) für eine Folge von Buchstaben und Ziffern, die mit einem Buchstaben beginnt
  - Unterscheidung von Groß- und Kleinbuchstaben hängt von gewählter Programmiersprache ab
- Verwendung ***einer vordefinierten Sammlung an Namen*** zur Beschreibung von Sprachkonstrukten (***if, then, else, while, etc.***)
  - *reservierte Schlüsselwörter* dürfen nicht als Bezeichner verwendet werden
  - *vordefinierte Bezeichner* (etwa zur Benennung von Bibliotheksfunktionen) können neu definiert werden

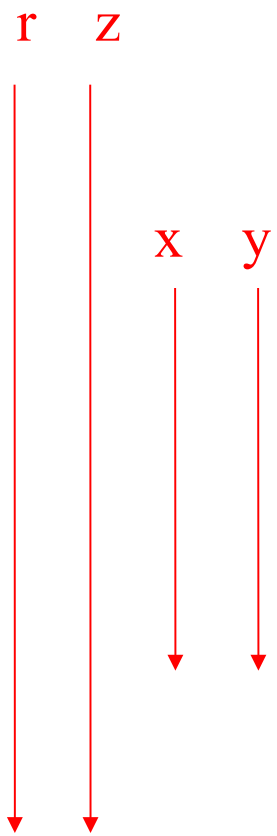
Namen: Gültigkeitsbereich und Lebensdauer

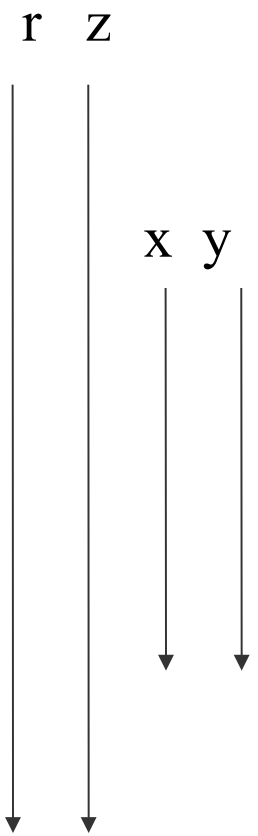
- ***Gültigkeitsbereich:*** Programmteil in dem auf einen Bezeichner zugegriffen werden kann
  - beginnt in prozeduralen Programmen oftmals mit Deklaration des Bezeichners
  - Gültigkeit eines Namens in blockorientierten Sprachen
    - beschränkt auf den Block in dem der Bezeichner definiert ist
    - sind geschachtelte Blöcke erlaubt, kann der Gültigkeitsbereich durch Neu-Definition in inneren Blöcken eingeschränkt werden
- ***Lebenszeit:*** Zeitintervall für das während der Laufzeit dem Bezeichner ein Speicherplatz zugeordnet ist

Beispiel: Gültigkeitsbereich und Lebensdauer

Gültigkeitsbereich *Lebensdauer*

```
void p (void)
{
     int r, z;
    ....
     {
          int x, y;
               x = 2;
               r = 1;
               y = 0;
               x  = x + y + r;
     }
     ....
}
```

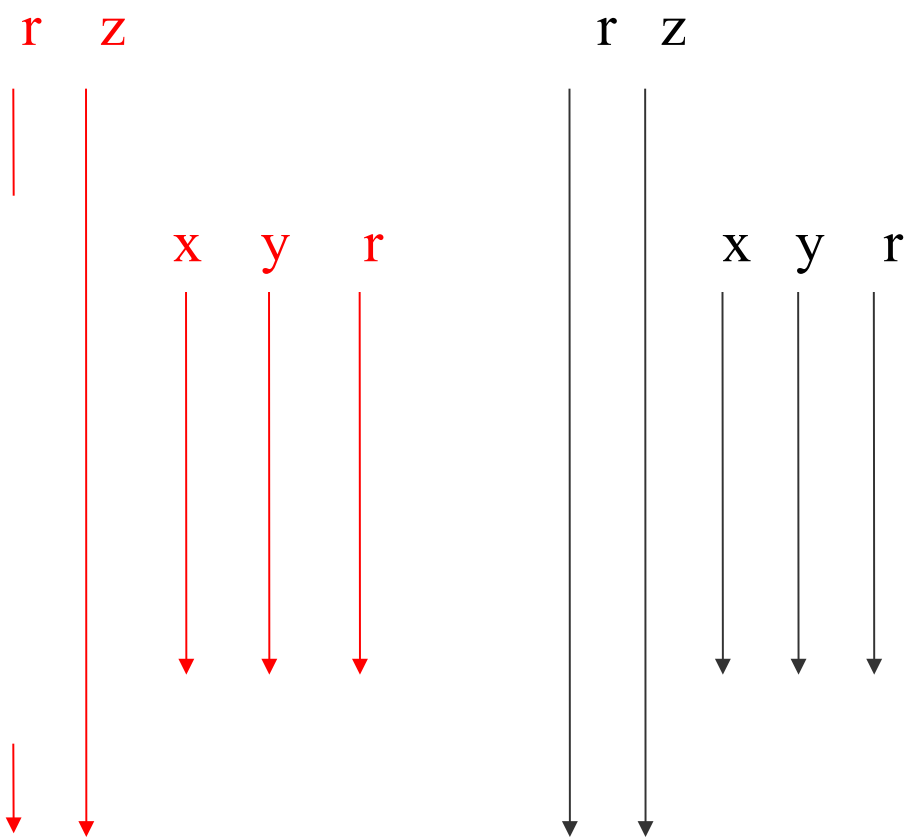




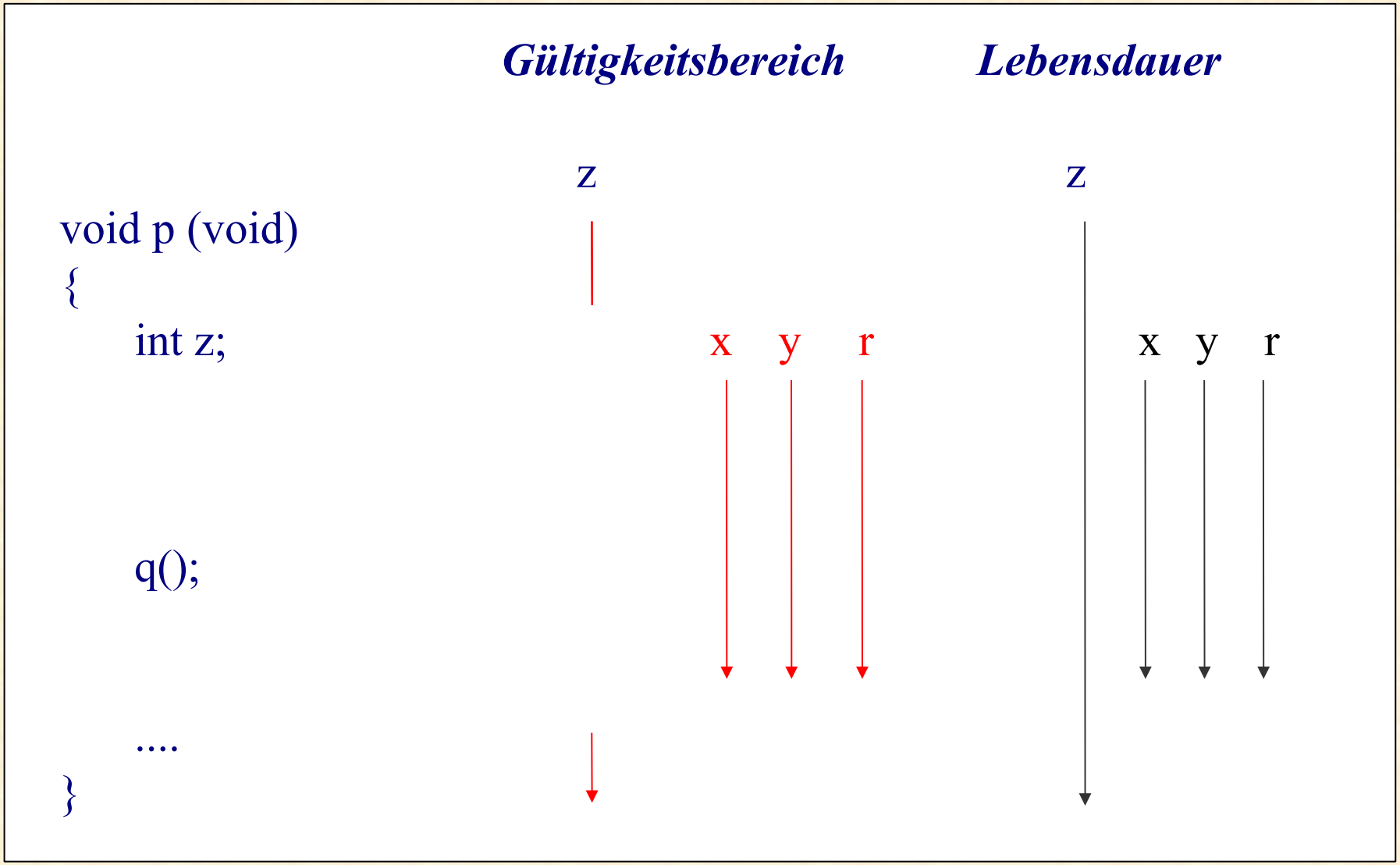
Beispiel: Gültigkeitsbereich und Lebensdauer

```
void p (void)
{
     int r, z;
    ....
     {

          int x, y, r;
               x = 2;
               r = 1;
               y = 0;
               x  = x + y + r;
     }
     ....
}
```



Beispiel: Gültigkeitsbereich und Lebensdauer



Datentypen

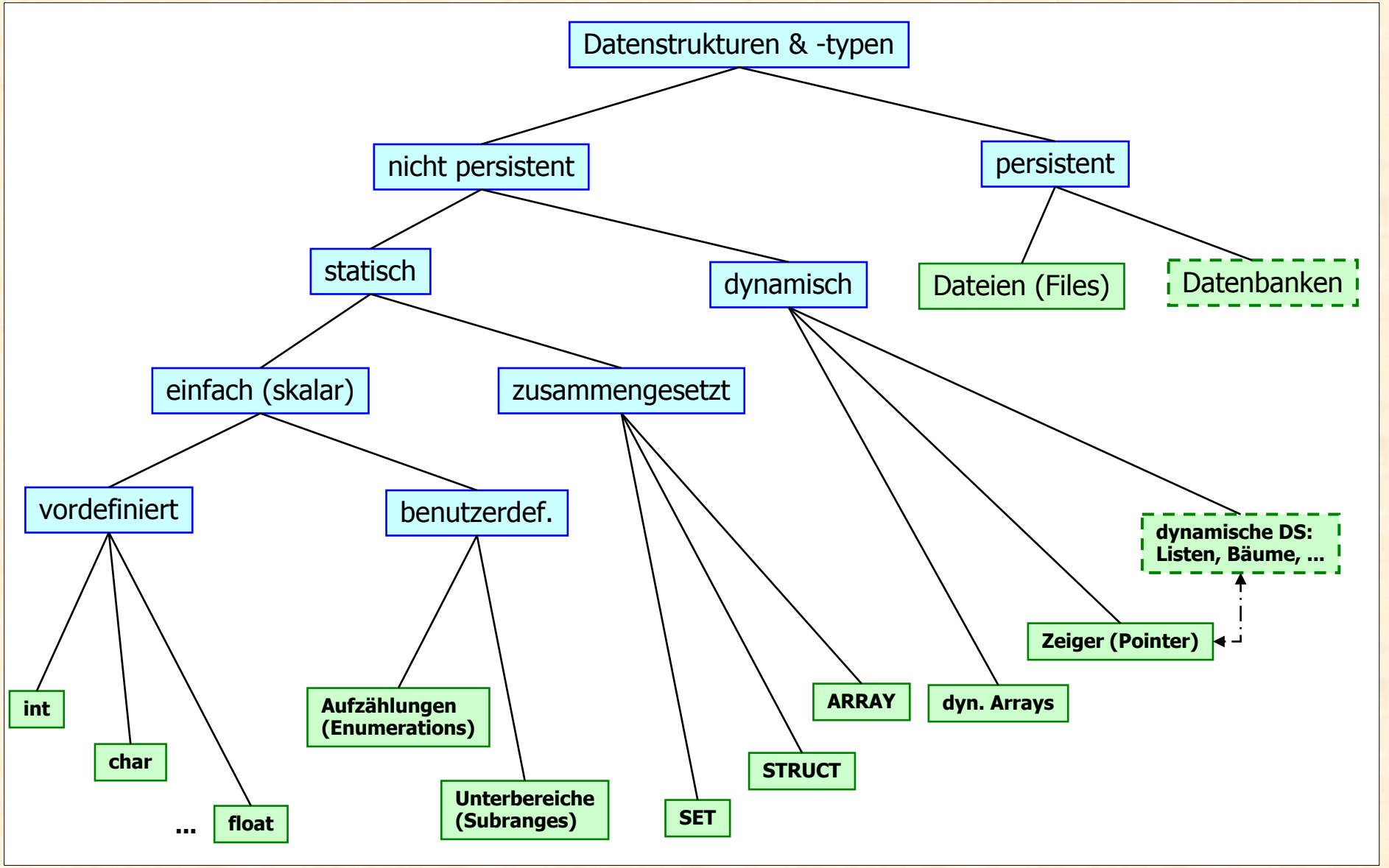
Verwendung von Datentypen: Allgemeines

- Ein ***Datentyp*** definiert
  - eine Menge von zugelassenen Werten, den ***Wertebereich***
  - ***Menge von Operationen***, die auf diesen Werten ausgeführt werden können.
- Ein Datentyp
  - kategorisiert die im Programm verwendeten Daten
    - Variablen werden immer unter Verwendung von einem Datentyp deklariert
    - eventuell auch initialisiert
    - bestimmt damit auch schon den Speicherbedarf
  - schränkt Interpretation und Verarbeitung der Daten ein
    - dies ist erwünscht!
  - ermöglicht semantische Korrektheit eines Programms zu überprüfen
    - Auffinden von nicht deklarierten Variablen
    - Gültigkeit von Ausdrücken
    - Anwendbarkeit von Operationen

Programmiersprachen: Typsysteme

- von einer Programmiersprache unterstützte Datentypen werden durch das ***Typsystem einer Sprache*** beschrieben
- Überprüfung der korrekten Verwendung des Typsystems gehört zur semantischen Analyse eines Programms (Phase: Typüberprüfung)
  - kann nicht syntaktisch durch die Grammatik beschrieben werden
  - wird für viele Sprachen (C, Pascal, etc.) vollständig zur Übersetzungszeit durchgeführt
  - zur Gewährleistung der Typsicherheit führen andere Sprachen (wie Java und C#) sowohl Überprüfungen zur Übersetzungs- als auch zur Laufzeit durch
- Sprachen heißen ***streng typisiert***, wenn alle Typfehler zur Übersetzungs- oder Laufzeit eines Programms erkannt werden
  - Ada und Java sind streng typisierte Sprache, C und C++ nicht

Datentypen: Kategorisierung



Vordefinierte skalare Datentypen

- zu jeder Programmiersprache gehört ein Satz vordefinierter Datentypen (in C: *char, short, int, long, long long int, float, double, long double*)
  - meist einfache, primitive, in sich unstrukturierte Typen
  - andere (zusammengesetzte) Datentypen können hieraus gebildet werden
- vordefiniert
  - direkt im Sprachumfang beinhaltet
  - „Struktur“ und Operationen müssen nicht erst programmiert werden
  - Wertebereich liegt (eventuell maschinenabhängig) bereits fest
  - viele zusätzliche vordefinierte Funktionen sind verfügbar
- nicht persistent
  - überleben nicht die Ausführungszeit des Programms
- statisch
  - „Struktur“ und Größe stehen fest: einzelnes Element bestimmten Typs
  - Speicherplatz wird abhängig vom Block angelegt und gelöscht

Benutzerdefinierte skalare Datentypen

- nicht vordefinierte einfache Typen, die vom Benutzer über statische Typkonstruktoren definiert werden müssen
  - ***der Wertebereich wird durch den Benutzer definiert***
  - keine eigentliche „Struktur“
  - Operationen oft abhängig vom Basistyp
- in vielen Programmiersprachen definierbar
  - Aufzählungstyp
  - Unterbereichstypen: Unterbereich eines existierenden vordefinierten Datentyps

```
enum Farbe {ROT, BLAU, GRUEN};
enum Lehrveranstaltung {VORLESUNG, UEBUNG, PRAKTIKUM};

enum Sprache c;
enum Lehrveranstaltung grundlagen_informatischer_Problemlösung;
```

Zusammengesetzte statische Datentypen

- zusammengesetzte Datentypen sind ein Datenkonstrukt, das aus einfacheren Datentypen besteht
  - Datensammlungen, Datenaggregate
- werden vom Benutzer, ***nach vorgegebenen „Strukturmustern“, die die Sprache unterstützt***, definiert
  - Wertebereich abhängig von den “aggregierten” Typen
  - Operationen auf Elementen definiert
  - Operationen auf zusammen gesetzten Datentypen:
    Variablendeklaration
    Konstruktion
    Projektion (Elementzugriff) schreibend und lesend

Zusammengesetzte statische Datentypen

Beispiel Array:

```
float ergebnis [365];
        ....
ergebnis[8] = 150191.8;
ergebnis[9] = 160250.9;
        ....
X = ergebnis[42];
```

- N gleichartige Datenobjekte
- Zugriff über numerischen Index
- Indizierung beginnt bei 0 (in C)
- Keine Bereichsüberprüfung des Index (in C)

Zusammengesetzte statische Datentypen

Beispiel Struktur/Record:

- Feste Anzahl
- (potentiell) verschieden-artiger Datenobjekte
- Zugriff über Eintragsname
- Überprüfung gültiger Eintragsnamen durch Compiler
- Speicherplatz für alle Einträge
- „Produkttyp“

```
struct datum {
  unsigned int tag;
  enum monat {
    Jan, Feb, ...
  } monat;
  unsigned int jahr;
};

struct datum geburtsdatum;
          ....
geburtsdatum.tag = 12;
geburtsdatum.monat = Feb;
geburtsdatum.jahr= 2000;
          ....
int x = geburtsdatum.tag;
```

Zusammengesetzte statische Datentypen

Beispiel Union/Variante:

- Feste Anzahl
- (potentiell) verschieden-artiger Datenobjekte
- Zugriff über Eintragsname
- Überprüfung gültiger Eintragsnamen durch Compiler
- Speicherplatz für größten Eintrag
- „Summentyp“

```
struct rectangle {
  int x,y;
  unsigned int xs, ys;
}

struct circle {
  int x,y;
  unsigned int radius;
}

union body {
  struct rectangle rect;
  struct circle circ;
}

union body b;

b.circ.radius = 42;
```

Zusammengesetzte statische Datentypen

Union:

- Woher weiß man, was man hat?
- Einbettung in weitere Struktur

```
struct body {
  enum {rectangle, circle} kind;
  union body body;
}

struct body b;

if (b.kind == circle) {
  b.body.circ.radius = 42;
}
```

Dynamische Datenstrukturen

- Dynamik betrifft
  - Struktur und/oder Größe
  - aber auch Zeitpunkt (und Kontrolle) des Anlegens und Löschens von Speicherplatz
- als Struktur nicht im Sprachumfang beinhaltet
  - ***die Struktur und alle Operationen auf der Struktur werden durch den Benutzer frei programmiert***
  - Wertebereich abhängig von den Elementen der Struktur
  - verwenden meist zusätzlich ***spezielle skalare Datentypen (Zeiger)*** um eine verzeigerte Struktur aufzubauen
- Beispiele:
  - dynamische Arrays
  - Stack, Liste, Baum, etc.

Persistente Datenstrukturen

- Datenhaltung erfolgt extern zum Programm
  - ***Daten überleben das Programm***
  - Daten haben eventuell schon vor dem Programm existiert
- Nutzung externer Infrastruktur:
  - Datenbanken
  - Dateien des OS